

MOATLY

Firmware — Technical Breakdown

A single Zephyr RTOS firmware image for the nRF5340, serving both the Premium and Simple hardware tiers via runtime capability detection. Covers BLE, the combination engine, the glow driver, signed DFU, and the attestation chain the backend relies on.

Prepared for: Firmware engineer and engineering lead, Moatly

Document version: 0.1 · Status: Draft for review

Date: April 2026

0. How to read this document

This is the firmware breakdown. The mobile-app Phase 3 breakdown specified the *app side* of the device integration — BLE pairing, combination-capture UIs, the composite identity code, the OTA initiation flow. This document specifies the **other side of every one of those conversations**: what runs on the nRF5340 in the plastic enclosure, signs the challenges, verifies the combinations, drives the glow, and accepts signed firmware updates.

The firmware is one codebase that serves two hardware tiers. Premium has a capacitive touch ring and an RGB LED halo; Simple has a single button and a single-colour LED. Underneath, both have the same SoC, the same secure element, and — critically — the **identical cryptographic story**. What differs is the input driver, the LED driver, and some details of the combination vocabulary. Runtime capability detection (§3.5) decides which drivers to enable.

The document is written for a single firmware engineer who will own the codebase end-to-end, with occasional pairing from the security engineer on the attestation and DFU work.

0.1 What this document assumes

- The reader has read the Build Brief v1.0, especially §4.3 (hardware / firmware stack), §5.5 (the Device), §6.4 (anti-Sybil and attestation).
- The reader has read the Moat App Phase 3 breakdown — the mobile side of the GATT profile, the combination capture, the glow trigger API, and the DFU flow.
- The reader has read the backend breakdown — specifically §9.1 (device attestation verification) and §4 (firmware manifest endpoint).
- The reader is comfortable with Zephyr, nRF Connect SDK, and the PSA Crypto API.

0.2 What this document does not cover

- Industrial design. The enclosures, the touch ring physical layout, the LED diffuser, the button feel. Owned by the designer.
- Electrical design and PCB layout. Owned by the hardware engineer.
- Factory production test firmware. A separate tiny image used at manufacture. Covered briefly in §10 but has its own test-engineering document.
- The app. Covered by the three app-phase breakdowns.
- The backend. Covered by the backend breakdown.

0.3 Document map

§	Contents	Primary reader
1	What the firmware is — platform, tiers, what it owns.	Firmware + lead
2	Decisions required before Week 1.	Firmware + security
3	Zephyr architecture — threads, subsystems, device tree, Kconfig.	Firmware
4	The BLE GATT profile — the complete characteristic surface.	Firmware + mobile

§	Contents	Primary reader
5	The combination engine — input capture to cryptographic unlock.	Firmware + security
6	Secure boot, attestation, and PSA Crypto key storage.	Firmware + security
7	Signed firmware updates (Nordic Secure DFU).	Firmware + security
8	The glow driver.	Firmware + designer
9	Power management — wake events, deep sleep, battery target.	Firmware + hardware
10	Factory provisioning and production vs. debug firmware.	Firmware + ops
11	Cross-cutting — memory budget, timing, security posture.	Firmware + lead
12	Testing — hardware-in-the-loop, unit, and security tests.	Firmware + QA
13	Implementation plan — 12 weeks synchronised with app Phase 3.	Lead + PM
14	Open questions.	Lead + founder
15	Post-launch lifecycle — field updates, forensics, revocation.	Firmware + SRE

1. What the firmware is

1.1 Hardware platform

Component	Choice and rationale
SoC	Nordic nRF5340. Dual-core Arm Cortex-M33 (application + network), integrated BLE 5.3 radio, TrustZone, Arm CryptoCell-312 secure element. Dual-core enables hard separation of untrusted application code from trusted crypto code.
Secure element	CryptoCell-312 on the nRF5340 network core. Hardware-isolated AES, SHA, ECC. Backs the PSA Crypto API.
OS	Zephyr RTOS. Nordic first-class support, open-source, Apache 2.0 licence-aligned with ADR-007.
Input (Premium)	8-electrode capacitive touch ring. Driver: nRF CAPSENSE via Zephyr sensor API.
Input (Simple)	Single hardware button on a GPIO with internal pull-up. Interrupt-driven.
Output (Premium)	Addressable RGB LED ring (WS2812-compatible), driven via SPI with DMA.
Output (Simple)	Single amber LED on a PWM GPIO.
Storage	Internal flash (1 MB). Zephyr settings subsystem with NVS backend for key-value persistence.
Charging	USB-C via a dedicated PMIC (handled outside firmware; firmware only reads state-of-charge over I2C).
Battery	300 mAh LiPo (target; exact capacity depends on enclosure). Firmware targets >30 days between charges.

1.2 Two tiers, one image

The brief (§4.3) calls for 'a single firmware image across Premium and Simple tiers'. This firmware achieves that via a startup probe that reads a single byte from OTP (One-Time-Programmable) memory written at manufacture, identifying the tier. Tier-dependent drivers are enabled conditionally; tier-dependent code paths branch cleanly on a static variable.

Consequences of the single-image choice:

- The codebase is slightly larger than a tier-specific image would be (we carry drivers for both the capacitive ring and the button). Flash budget accommodates this comfortably.
- Every OTA update ships to every device regardless of tier. Simpler ops, simpler signing story.
- A bug in tier-A-only code is still carried on tier-B devices. Mitigated by the discipline that tier-specific code is firewalled behind the hardware-capability check.

1.3 What the firmware owns

- **Being a BLE peripheral** discoverable by the paired phone, advertising when appropriate, accepting connections from the known central.

- **Signing the challenges** the app forwards from the backend, using the device's hardware-backed private key.
- **Verifying the user's combination** against its stored expected value, and releasing the key for a single signature on match.
- **Glowing when told to** via the GATT write from the app, in one of five defined patterns.
- **Accepting firmware updates** if and only if they are signed by the pinned key and have a strictly greater version number.
- **Sleeping efficiently** so the 30-day battery target is met.
- **Self-wiping under threshold conditions** — 20 consecutive combination failures zeroes the signing key and forces re-pair.

1.4 What the firmware does not do

- The firmware does not decrypt or see any Moat content. Ever. It signs challenges; it does not see what those challenges are used to authenticate.
- The firmware does not know about the user's email, Moat membership, Major Statements, rings, or any other product concept. Those are all in the app.
- The firmware does not call the internet. It talks only to the paired phone over BLE.
- The firmware does not log to any external destination. In production builds it does not log at all; in debug builds it logs to RTT (Segger's on-chip debugger).
- The firmware has no phone-home telemetry, no crash reporting, no beacons. Period.

2. Decisions required before Week 1

Nine decisions. Default recommendation, trade-off summary, owner.

2.1 Zephyr version — LTS or current

Recommendation: Zephyr 3.7 LTS.

Zephyr's LTS releases receive security fixes for 2.5 years. Current is moving faster than our hardware cadence justifies. Nordic's nRF Connect SDK (which bundles Zephyr) also tracks LTS closely; picking the LTS keeps us on the well-tested Nordic integration.

2.2 SoC — nRF52840 or nRF5340

Recommendation: nRF5340.

The Build Brief (§4.3) names both ('nRF52840 (BLE 5, secure element optional via nRF5340)'), but the *Key storage* row then says 'nRF5340 CryptoCell + PSA Crypto API: Hardware-isolated key storage; required for the attestation story.' The attestation story is non-negotiable; the CryptoCell is how we get it; the CryptoCell is on the 5340. Decision follows.

The cost delta at our volumes (~10k units for the first year) is £1–2 per unit. Worth it for hardware key isolation we can point at during external audit.

2.3 Build system — west + CMake (Zephyr default) or bespoke

Recommendation: west + CMake as Zephyr ships it. No bespoke wrapper.

west is Nordic's and Zephyr's recommended multi-repo workflow. It handles the ~20 repos that make up a Zephyr build (zephyr itself, the modules, the HAL, mbedtls, picolibc, etc.). A bespoke wrapper saves typing and costs auditability.

2.4 Secure boot — MCUboot or custom

Recommendation: MCUboot. Nordic-supported configuration.

MCUboot is the open-source bootloader used throughout the Zephyr ecosystem. It handles signed firmware verification at boot, dual-slot storage for swap-based upgrades, and rollback on boot failure. Nordic's default MCUboot configuration on nRF5340 is production-ready.

2.5 Attestation key algorithm

Recommendation: Ed25519 via PSA Crypto. Hardware-backed on CryptoCell-312.

Ed25519 is fast, compact, widely audited, deterministic (no nonce failure risk), and well-supported by the PSA Crypto API on nRF53 CryptoCell. The alternative — ECDSA on P-256 — is also fine but a larger attack surface for implementation errors and a little slower.

2.6 DFU transport — Nordic Secure DFU or MCUboot SMP

Recommendation: MCUboot + SMP (Simple Management Protocol) over BLE.

Nordic's legacy Secure DFU is deprecated in favour of MCUboot+SMP for modern Zephyr-based projects. MCUboot+SMP is the current Nordic recommendation, is what MCUboot

integrates natively with, and has a mature mobile client (mcumgr) we can reference from our app's DFU module.

2.7 Combination storage format

Recommendation: Store an HKDF-derived key that can only be reconstructed by re-entering the combination. Do not store the combination itself, not even hashed.

The combination derives a 256-bit key via PBKDF2 (with a per-device salt in flash). That derived key is the PSA unlock credential for the attestation private key. Correct combination → correct derived key → PSA unlocks the signing key → signature returned. Wrong combination → wrong derived key → PSA refuses. The firmware never stores the combination in any form — not the raw input, not a hash, nothing.

2.8 Logging

Recommendation: Zephyr's logging subsystem, compiled out entirely in production builds. In debug builds, output to RTT only (no UART).

A Kconfig flag (CONFIG_MOATLY_PRODUCTION_BUILD) controls the whole logging subsystem. In production, the logging module is not linked — string literals for log messages are not present in the binary. This prevents reverse-engineers from learning internal state names by reading flash.

2.9 Tier detection — hardware strap, OTP, or firmware flag

Recommendation: OTP (One-Time-Programmable) byte, written at factory provisioning.

An OTP byte is tamper-evident (it can be written once and never changed) and is read at boot. A hardware strap (GPIO tied high/low) would work but adds a routing constraint for the hardware engineer and can be fooled in a field-modification scenario. A flash-stored flag is weakest — it can be re-written by malicious firmware before the tier-dependent driver loads.

2.10 Summary table

Decision	Default recommendation	Owner
Zephyr version	3.7 LTS	Firmware
SoC	nRF5340 (CryptoCell required)	Firmware + hardware
Build system	west + CMake (stock Zephyr)	Firmware
Secure boot	MCUboot	Firmware + security
Attestation key algo	Ed25519 via PSA Crypto	Security
DFU transport	MCUboot + SMP over BLE	Firmware + security
Combination storage	Never stored; derives PSA unlock key	Security
Logging	RTT in debug; absent in production	Firmware
Tier detection	OTP byte at factory	Firmware + ops

3. Zephyr architecture

3.1 Repository layout

```

moatly-firmware/
├── west.yml
├── app/
│   ├── CMakeLists.txt
│   ├── prj.conf
│   ├── prj.debug.conf
│   ├── boards/
│   │   ├── moatly_premium.dts
│   │   ├── moatly_simple.dts
│   │   └── moatly_common.dtsi
│   └── src/
│       ├── main.c
│       ├── tier.c
│       ├── ble/
│       │   ├── advertise.c
│       │   ├── connect.c
│       │   └── gatt.c
│       ├── attest/
│       │   ├── psa_keys.c
│       │   ├── challenge.c
│       │   └── factory_cert.c
│       ├── combo/
│       │   ├── capture_ring.c
│       │   ├── capture_button.c
│       │   ├── vocabulary.c
│       │   ├── derive.c
│       │   └── lockout.c
│       ├── glow/
│       │   ├── driver_ring.c
│       │   ├── driver_led.c
│       │   ├── patterns.c
│       │   └── scheduler.c
│       ├── dfu/
│       │   └── mcumgr.c
│       ├── power/
│       │   ├── sleep.c
│       │   └── battery.c
│       └── storage/
│           ├── settings.c
│           └── factory.c
├── tests/
│   ├── unit/
│   └── hil/
└── tools/
    ├── sign_manifest.py
    └── factory_provision.py

```

west manifest repo
pulls Zephyr 3.7 LTS, Nordic HAL, MCUboot
our application
Kconfig for production
Kconfig for debug builds
device tree for Premium hardware
device tree for Simple hardware
shared device tree fragment
init + application thread spawn
OTP read + runtime capability gate
GATT service definitions
PSA Crypto bindings for the signing key
challenge handler
factory cert load + expose
Premium – capacitive gestures
Simple – press pattern
shared gesture → integer mapping
PBKDF2 → PSA unlock credential
retry + lockout state machine
Premium – RGB WS2812 via SPI/DMA
Simple – single LED PWM
5 event patterns
state machine
SMP handlers on top of MCUboot
deep-sleep coordinator
PMIC I2C read
Zephyr settings subsystem bindings
factory-provisioned blobs
Zephyr ztest unit tests
hardware-in-the-loop
offline signing (run on air-gapped host)
attached at manufacture

3.2 Thread model

Zephyr's cooperative + preemptive scheduler gives us typed threads with priorities. We run five threads beyond the kernel's own:

Thread	Priority	Responsibility
main	K_PRIO_P REEMPT(7)	Boot sequence, tier detection, system init, then sleeps forever.
ble_thread	K_PRIO_C OOP(5)	Owns BLE stack interactions. Receives events from Zephyr's BT subsystem.
combo_thread	K_PRIO_P REEMPT(6)	Input capture → combination verification. Runs when input is pending.
glow_thread	K_PRIO_P REEMPT(8)	Glow pattern state machine. Runs the active pattern at the correct cadence.
dfu_thread	K_PRIO_C OOP(5)	MCUboot SMP handlers. Active only during a firmware update.
power_thread	K_PRIO_P REEMPT(9)	Sleep coordinator. Monitors activity; requests deep sleep when idle.

Threads communicate via message queues (k_msgq) and semaphores (k_sem). No shared mutable state outside of these primitives.

3.3 Subsystems

Zephyr ships the following subsystems, which we use as-is:

- **Bluetooth** — nRF Connect SDK's BT stack, configured as peripheral-only.
- **PSA Crypto** — Arm's PSA Crypto API backed by CryptoCell-312 via Zephyr's TF-M integration.
- **Settings** — key/value persistence on flash with wear-levelling via NVS.
- **MCUboot + SMP** — bootloader and management protocol.
- **Power management** — PM subsystem for device and system power states.
- **Logging** — Zephyr's structured logger (disabled in production).

3.4 Device tree

Two board definitions share a common fragment:

```
/* moatly_common.dtsi - shared */
/ {
    chosen {
        zephyr,console = &uart0;          /* only present on dev boards */
        zephyr,flash = &flash0;
        zephyr,sram = &sram0;
        zephyr,code-partition = &slot0_partition;
    };
};
```

```

moatly_tier: moatly-tier {
    compatible = "moatly,tier-otp";
    otp-address = <0x10001080>;    /* nRF5340 UICR CUSTOMER reg */
};

&flash0 {
    partitions {
        compatible = "fixed-partitions";
        boot_partition: partition@0 { reg = <0x00000 0x10000>; };
        slot0_partition: partition@10000 { reg = <0x10000 0x70000>; };
        slot1_partition: partition@80000 { reg = <0x80000 0x70000>; };
        settings_partition: partition@f0000 { reg = <0xf0000 0x10000>; };
    };
};
/* moatly_premium.dts */
#include "moatly_common.dtsi"

/ {
    aliases {
        capsense0 = &capsense;
        led-strip = &led_strip;
    };

    capsense: capsense@0 {
        compatible = "nordic,nrf-cap-touch";
        electrodes = <8>;
        gpios = <&gpio1 0 0>, <&gpio1 1 0>, ...;
    };

    led_strip: ws2812 {
        compatible = "worldsemi,ws2812-spi";
        spi = <&spi2>;
        num-leds = <16>;
    };
};
/* moatly_simple.dts */
#include "moatly_common.dtsi"

/ {
    aliases {
        button0 = &button;
        led0 = &amber_led;
    };

    buttons {
        button: button_0 {
            gpios = <&gpio0 11 (GPIO_PULL_UP | GPIO_ACTIVE_LOW)>;
            label = "Moatly button";
        };
    };

    pwmleds {

```

```

compatible = "pwm-leds";
amber_led: amber_led_0 {
    pwms = <&pwm0 0 PWM_MSEC(20) PWM_POLARITY_NORMAL>;
    label = "Amber LED";
};
};
};

```

3.5 Runtime capability detection

At boot, main.c reads the OTP tier byte and sets a global tier value. Every driver and every code path that is tier-dependent branches on this.

```

/* src/tier.c */
enum moatly_tier {
    MOATLY_TIER_UNKNOWN = 0x00,
    MOATLY_TIER_PREMIUM = 0x01,
    MOATLY_TIER_SIMPLE = 0x02,
};

static enum moatly_tier s_tier = MOATLY_TIER_UNKNOWN;

int moatly_tier_init(void) {
    uint8_t otp_val;
    int ret = otp_read(CONFIG_MOATLY_TIER_OTP_OFFSET, &otp_val, 1);
    if (ret < 0) {
        /* Unprovisioned. Refuse to boot further than a diagnostic state. */
        return -EFAULT;
    }
    if (otp_val != MOATLY_TIER_PREMIUM && otp_val != MOATLY_TIER_SIMPLE) {
        return -EINVAL;
    }
    s_tier = (enum moatly_tier)otp_val;
    return 0;
}

enum moatly_tier moatly_tier_get(void) { return s_tier; }

```

4. The BLE GATT profile

The GATT profile is the contract between firmware and app. The app-side definitions live in the Phase 3 breakdown §7.5; this section specifies the firmware side with matching UUIDs.

4.1 The Moatly primary service

Service UUID: 5f6d6174-6c79-4d4f-0001-000000000000

Characteristics, in the order the app probes them:

UUID suffix	Purpose	Properties
...000000000010 Attestation	Read factory cert + hw_pubkey	READ
...000000000011 Challenge	App writes a challenge nonce	WRITE
...000000000012 Signature	Firmware writes signed response	READ, NOTIFY
...000000000020 Glow	App writes a glow event code	WRITE
...000000000030 Status	Device status (tier, battery, lockout)	READ, NOTIFY
...000000000040 DFU Ctl	(Proxy to MCUboot SMP.)	WRITE, NOTIFY
...000000000041 DFU Data	(Proxy to MCUboot SMP.)	WRITE

4.2 Characteristic details

4.2.1 Attestation (read)

Returns a serialised struct containing:

- 32 bytes — Ed25519 public key (the device's attestation pubkey).
- 2 bytes — length prefix for factory cert.
- N bytes — factory certificate (signed by Moatly's factory CA).
- 1 byte — tier identifier (0x01 Premium, 0x02 Simple).
- 1 byte — firmware feature-flags bitmap (reserved for future).

Total typical size: ~250 bytes. Fits in a single ATT MTU exchange post-negotiation.

4.2.2 Challenge (write)

App writes a 32-byte nonce from the backend. The firmware queues a combination-verify request on the combo_thread. The firmware does not respond yet — a signature only emerges after the user enters their combination correctly.

4.2.3 Signature (read, notify)

Once the user's combination succeeds and the PSA key unlocks, the firmware signs (challenge_nonce || device_pubkey_hash) with Ed25519 and notifies. The app reads the 64-byte signature. On a failed combination (either wrong input or the combination-capture is cancelled) the characteristic emits a single-byte notification:

- 0x01 — signature available to be read.
- 0x02 — combination failed, try again.
- 0x03 — locked out, see Status for duration.

- 0x04 — user cancelled.

4.2.4 Glow (write)

Single-byte event code. Mapped to one of the 5 patterns from brief §5.5.4:

```
0x01 – major_statement      (slow amber pulse, until app opens or 30 min)
0x02 – cycle_begin          (soft fade-in/out once, 8 seconds)
0x03 – ring_completed       (gold shimmer once, 4 seconds)
0x04 – low_battery          (two red blinks, once) [firmware-initiated]
0x05 – stop                 (immediate silent state)
Others: ignored silently.
```

4.2.5 Status (read, notify)

Packed struct, 8 bytes:

```
uint8_t  firmware_major;
uint8_t  firmware_minor;
uint8_t  firmware_patch;
uint8_t  tier;                // 0x01 Premium, 0x02 Simple
uint8_t  battery_percent;    // 0–100
uint8_t  combination_attempts_remaining_before_lockout;
uint16_t lockout_remaining_seconds; // 0 if not locked out
```

Updates via notify on change of any field.

4.2.6 DFU Ctl / DFU Data

Pass-through to MCUboot's SMP handlers. The firmware does not implement DFU directly; it delegates to mcumgr. Covered in §7.

4.3 Advertising

- Advertising is connectable, undirected, in BLE 5 Extended Advertising PDUs (to pack more data).
- Interval 200 ms when expecting a connection; 1000 ms in the 'paired but idle' state.
- Advertising payload includes the Moatly service UUID (truncated), a random-but-stable `display_pseudonym_bytes` that the app uses to render the 'Device — Crane Juniper' label at pairing time.
- RSSI-based wake: the firmware listens for advertisements from the paired phone; on receiving a signal above -70 dBm it transitions to full-connection-ready state. Below -85 dBm for 10 consecutive seconds → back to idle advertising.

4.4 Connection parameters

- Connection interval: 30 ms (fast enough for responsive glow trigger + DFU throughput).
- Slave latency: 4 (we can skip 4 connection events to save power when idle).
- Supervision timeout: 4 seconds.
- MTU negotiated to 247 (maximum for BLE 5 ATT). Required for the attestation read to complete in one round.

4.5 Pairing / bonding

We do **not** use standard BLE pairing (no passkey entry). The security model of Moatly is attestation-based: the app verifies the device's factory cert and challenge signature; the device accepts connections from any central that is able to make a connection. There is no PIN, no passkey, no shared secret negotiated at pair time.

Why no BLE bonding?

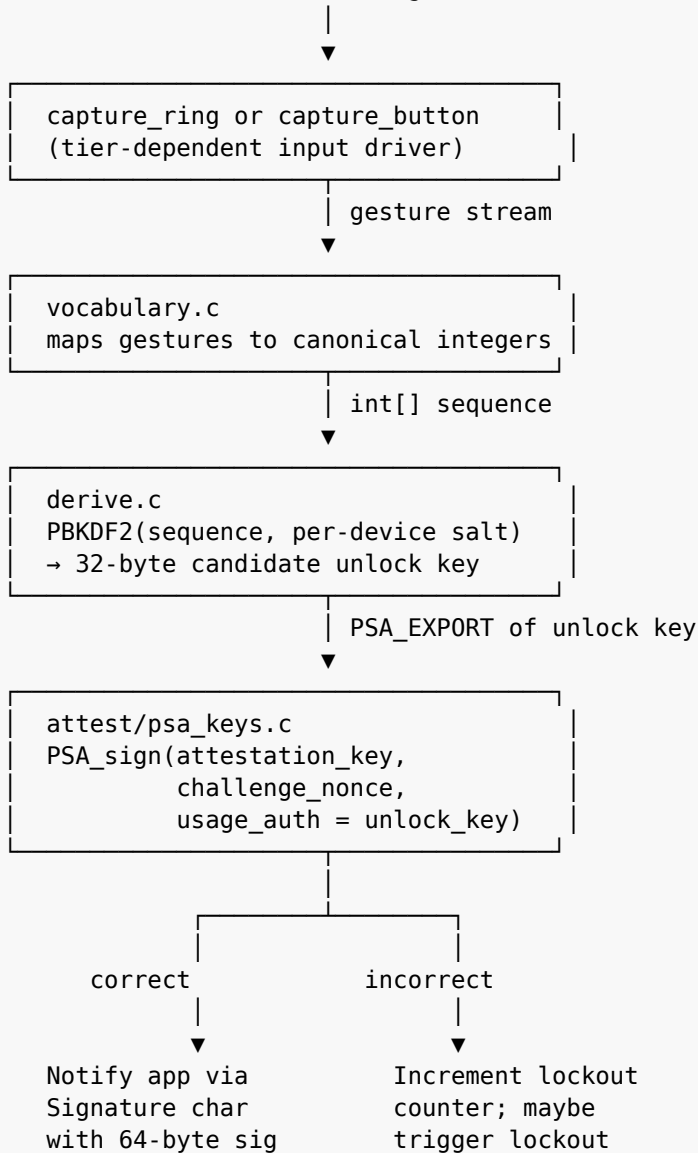
Standard BLE bonding stores a shared link key on both sides. This makes the link encrypted at the BLE layer — but the content we exchange is already an attested signature over a server-supplied nonce; the BLE-layer encryption adds nothing. Skipping bonding removes a state machine (and a failure mode: bond loss requiring re-pairing) without weakening security. This is an important choice to surface to the external security auditor.

5. The combination engine

The combination engine is the firmware's most intricate subsystem. It captures user input, turns that input into a deterministic derived key, uses that key to unlock the PSA-protected attestation private key for one signature, and updates retry / lockout counters.

5.1 The big picture

User enters combination on ring (Premium) or button (Simple).



5.2 Input capture — Premium (capacitive ring)

The 8-electrode ring samples at 60 Hz. Each sample yields 8 values; we compute the centroid of activated electrodes when any is above a touch threshold. Gesture recognition runs on the sample stream:

- Stable centroid on one electrode for <250 ms → tap-at-position (8 possible positions).

- Stable centroid on one electrode for ≥ 800 ms → long-press.
- Centroid moves clockwise through ≥ 3 electrodes within 500 ms → swipe CW.
- Same anticlockwise → swipe CCW.

Total gesture vocabulary for Premium: 10 distinct gestures (8 tap positions + CW + CCW + long-press = 11; tap + long-press are differentiated by duration on the same electrode). In practice we use *8 tap positions + swipe_CW + swipe_CCW + long_press = 11 gestures mapped to integers 0..10*.

The user's onboarding pattern is 5–7 of these gestures.

5.3 Input capture — Simple (press pattern)

The single button captures press duration and inter-press pauses. We quantise:

- **short press:** 50–300 ms hold → '.'
- **long press:** 400–1200 ms hold → '-'
- **short pause:** 150–500 ms gap → ';' (optional)
- **long pause:** 600–1500 ms gap → ',' (optional)

Length 8–14 tokens total. Pauses carry information but are optional — the user can enter 'short-short-long-short' without explicit pauses if they choose. The capture engine emits tokens on press-up events and infers pauses from inter-press intervals.

5.4 Gesture → integer mapping

Before the PBKDF2 step, the gesture stream is canonicalised. This matters because the timing values we captured are variable within tolerance bands (§1.6 of Phase 3). The canonicalisation rounds timing to 50 ms buckets and produces a stable integer sequence:

```
/* Canonicalised sequence, 2 bytes per step:
 *   byte 0: gesture_kind (tap_pos, swipe_cw, swipe_ccw, long, short_press,
 *                       long_press, pause_short, pause_long)
 *   byte 1: timing_bucket since previous gesture (0..255, each = 50 ms)
 *
 * This sequence is what we feed into PBKDF2.
 */
```

5.5 Derivation — PBKDF2 to unlock key

Per-device salt:

Written once at factory provisioning into a flash-protected slot.
32 bytes, generated by the factory provisioning tool from /dev/urandom.

PBKDF2 parameters:

PRF: HMAC-SHA-256
Salt: per-device salt
Iterations: 100,000
Output: 32 bytes (the unlock key)

The unlock key is then presented to PSA Crypto as the usage authentication for the attestation private-key slot.

Derivation takes approximately 500 ms on the CryptoCell at our

iteration count. This is visible to the user as the pause between "finished entering combination" and "device signs" – tuned so that it feels deliberate rather than slow.

5.6 Verification — PSA Crypto

The attestation private key is imported at factory provisioning with the PSA usage policy `PSA_KEY_USAGE_SIGN_HASH` plus a mandatory unlock credential. `PSA_sign_hash()` succeeds only when the caller supplies the unlock credential matching the expected unlock-key binding.

Concretely: we store a 32-byte HMAC-SHA-256(`real_unlock_key`, `'moatly.combo.v1'`) in flash. On a sign attempt we:

- Derive a candidate unlock key from the just-entered combination (§5.5).
- Compute HMAC-SHA-256(candidate, `'moatly.combo.v1'`).
- Compare to stored value with constant-time compare.
- On match: import candidate into PSA as unlock credential; call `psa_sign_hash()`; return signature.
- On mismatch: increment lockout counter; do not touch the PSA key.

5.7 Lockout policy

Enforced in firmware (brief §5.5). Written to flash settings to survive resets.

Consecutive failures	Lockout	Notes
1–4	None	Silent; user continues.
5	60 seconds	Status notify reports <code>lockout_remaining_seconds</code> .
10	10 minutes	Same.
15	24 hours	Same. Firmware resists clock-forward by tracking boot-relative time.
20	Permanent	Zero the PSA signing key. Zero the unlock-HMAC value. Require seed-phrase-based re-pair.

A successful signature resets the counter to zero.

5.8 Combination setup — onboarding flow

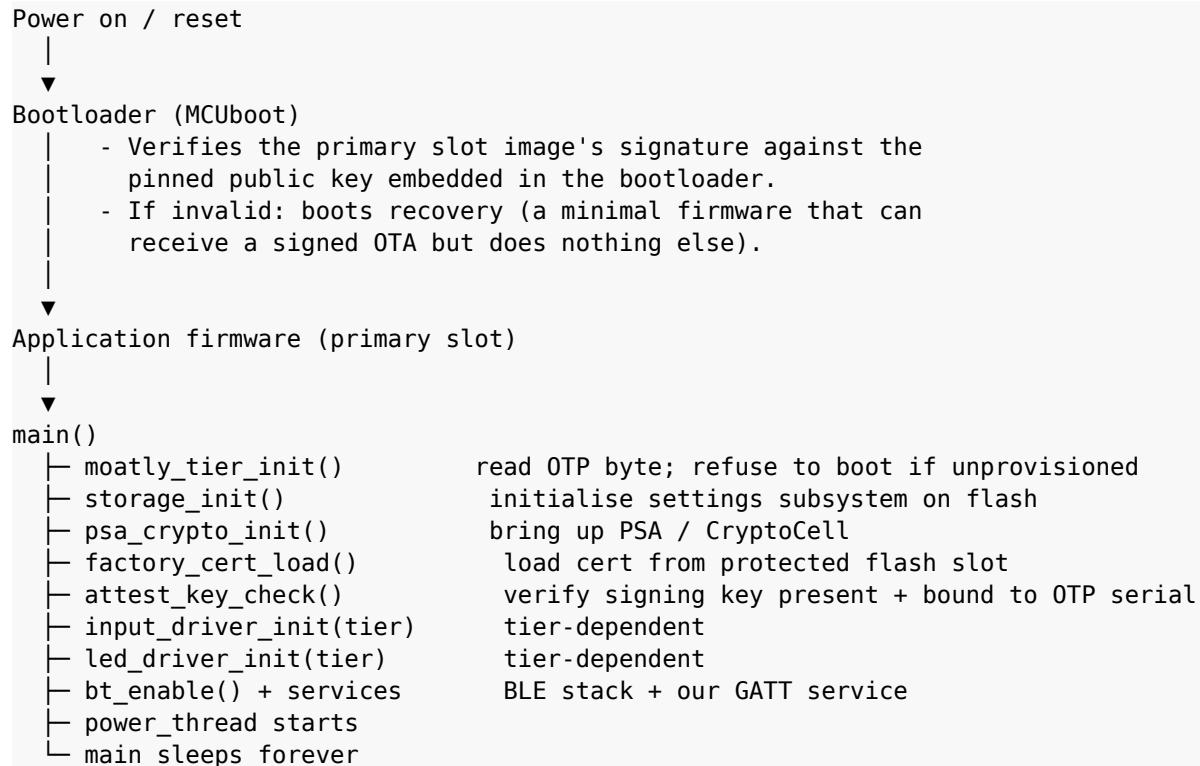
At first pair, the firmware is in 'no combination set' state. The app initiates combination setup by writing to a dedicated GATT characteristic (not listed in §4.1 because it is available only in this single state). The firmware:

- Accepts three consecutive entries from the app (the user enters the same combination three times on the device).
- Verifies all three canonicalise to the same sequence.
- Derives the unlock key via PBKDF2.
- Stores HMAC-SHA-256(`unlock_key`, `'moatly.combo.v1'`) to flash.

- Imports the attestation signing key (generated during factory provisioning) under that unlock credential.
- Leaves the 'no combination set' state; the setup characteristic is no longer available.

6. Secure boot, attestation, and PSA Crypto

6.1 Boot sequence



6.2 The attestation private key

- **Generated at factory provisioning** on the device itself, using `PSA_generate_key` with `PSA_KEY_TYPE_ECC_KEY_PAIR` and `PSA_ALG_PURE_EDDSA`.
- **Stored in CryptoCell-312's secure key store.** The raw key bytes never leave the CryptoCell.
- **Usage policy: SIGN_HASH only.** No export, no wrap, no copy.
- **Authentication required:** the unlock credential derived from the combination. PSA refuses to sign without it.
- **The public half** is exposed via the Attestation characteristic.

6.3 The factory certificate

A cert issued by Moatly's factory CA at manufacture time. Contains:

- Device serial number (also burned into OTP).
- Tier (Premium or Simple).
- Device public key.
- Factory timestamp.
- Ed25519 signature from the factory CA's private key.

Stored in a read-only flash slot written at provisioning. The app verifies the cert chains to our pinned factory CA (Phase 3 §4.3 of the app breakdown).

6.4 MCUboot configuration

- Signature algorithm: Ed25519.
- Image-slot layout: two slots of 448 KB each (primary at 0x10000, secondary at 0x80000). Swap-using-move upgrade strategy.
- Rollback counter: monotonic. Every OTA increments it. An OTA image with a lower counter than the installed one is refused at the bootloader level.
- Signing key pinned in the bootloader's .text segment (immutable post-provisioning).

6.5 Key zeroisation

On 20 consecutive combination failures (§5.7), the firmware zeroes:

- The attestation signing key in PSA storage (via `psa_destroy_key`).
- The unlock-HMAC stored in flash settings.
- The retry counter (reset to 0 so a re-paired device starts fresh).

After zeroisation the device reports 'no combination set' on next connection, and the app takes it through the re-pair flow from the seed phrase (Phase 3 §7.8).

7. Signed firmware updates

7.1 The pipeline

1. Moatly's release engineer builds a firmware image.
2. On an air-gapped signing host, the OTA private key signs a manifest that includes:
 - firmware version (monotonic)
 - image size + hash
 - rollback counter (monotonic, $\geq$ current device's counter)
 - release timestamp
 - Ed25519 signature over all of the above
3. Signed image + manifest published to the backend (/v1/firmware/manifest/:version and S3-hosted binary).
4. App checks the backend for an available update (Phase 3 §7.14).
5. User taps 'update' in DeviceSettings.
6. App verifies the manifest against its pinned OTA pubkey BEFORE any BLE transfer.
7. App uploads the image chunk-by-chunk over BLE via MCUboot SMP.
8. Bootloader verifies the image signature + rollback counter on next reset.
9. If verification passes: swap + boot new firmware.
10. If verification fails OR new firmware fails to run: bootloader reverts to the previous slot automatically.

7.2 The OTA signing key

- **Held in two-of-two offline custody** — security engineer and founder, on an air-gapped machine.
- Never resides on any internet-connected system.
- The public half is embedded in the bootloader at build time. Rotating the signing key requires a firmware release that updates the bootloader-embedded public key.

7.3 Rollback protection

The rollback counter is stored in a dedicated NVS record, protected by the CryptoCell's secure storage. Attempts to downgrade to an older-counter image are refused at the bootloader level even if the signature is valid.

7.4 Recovery firmware

If the primary slot image fails to boot (crash loop on startup) MCUboot falls back to a minimal recovery firmware held in the bootloader partition. Recovery firmware can:

- Advertise BLE as 'Moatly Recovery'.
- Accept a new signed OTA via MCUboot SMP.
- Nothing else — not attestation, not combination, not glow.

This gives us a 'brick recovery' path: a user with a wedged device can re-install firmware by triggering recovery (press-and-hold the button for 10s on Simple; long-press the ring on Premium) and running the update flow.

8. The glow driver

8.1 Two drivers, one pattern language

Premium has an addressable 16-LED RGB ring; Simple has one amber LED. The pattern engine is tier-agnostic and emits 'intensity-at-time-t' instructions. A small per-tier mapping layer turns those instructions into driver calls — WS2812 colour-frame writes on Premium, PWM duty cycle on Simple.

8.2 The five patterns

Event	Pattern on Premium	Pattern on Simple
major_statement	Slow amber pulse, 0.5 Hz. All 16 LEDs fade synchronously. Amber RGB $\approx$ (#FFA500).	Slow amber pulse at same 0.5 Hz cadence on the single LED.
cycle_begin	Soft fade-in to full amber, hold 2s, fade-out. Total 8 seconds.	Same, on the single LED.
ring_completed	Gold shimmer: each LED flickers with randomised mild amplitude, for 4 seconds.	Single fade up to warmer tone (full brightness amber), then down.
low_battery	Two quick red blinks on all LEDs.	Two quick bright blinks (amber, since no colour control).
stop	Immediate off.	Immediate off.

8.3 The state machine

```
enum glow_state {
    GLOW_IDLE,
    GLOW_PATTERN_ACTIVE,
};

struct glow_pattern {
    uint8_t event_code;
    uint32_t started_at_ms;
    uint32_t duration_ms;          /* 0 = run until stopped */
    void (*step_fn)(uint32_t elapsed_ms);
};

/* glow_thread loop: */
while (true) {
    struct glow_pattern p;
    if (k_msgq_get(&glow_queue, &p, K_FOREVER) == 0) {
        if (p.event_code == GLOW_STOP) {
            led_all_off();
            continue;
        }
        uint32_t start = k_uptime_get();
        while ((p.duration_ms == 0 || k_uptime_get() - start < p.duration_ms)
            && !stop_requested) {
```

```

    p.step_fn(k_uptime_get() - start);
    k_msleep(16);    /* ~60 Hz update for smooth fades */
}
led_all_off();
}
}

```

8.4 What the glow driver does not do

- No per-message glow. Only Major Statements, cycle begin, ring completion, and low battery trigger glow. Ordinary whispers and witnesses do not.
- No stacking. A new glow event cancels the previous one; we do not queue multiple events.
- No 'breathing' idle animation. The resting state of the device is dark.
- No per-user colour customisation. The glow is what it is.

8.5 Accessibility

- Pulse frequencies are kept below 3 Hz to avoid photosensitivity triggers (the medically dangerous band is 3–55 Hz).
- Colours chosen for AA contrast against dark enclosure backgrounds on a colour-vision-deficient spectrum (we rely on intensity, not hue, for communicating meaning on Premium; Simple uses intensity alone).
- Users who want the glow silenced entirely can do so via the app's Device Settings; the glow module accepts a 'mute all' override.

9. Power management

9.1 The budget

Target: >30 days between charges at 'typical' use. Typical is modelled as:

- 5 combination entries per day (each ~3 s active + 500 ms PBKDF2).
- 10 BLE connection events per day (glow triggers + status reads), each ~200 ms.
- 1 firmware update per month (a few minutes, negligible amortised).
- Otherwise: deep sleep, with BLE advertising at 1 Hz.

9.2 Power states

State	Current draw	Wake sources	What's running
Active	~6 mA	Always on	CPU active, BLE connected, LEDs possibly on, input driver sampling.
Idle-connected	~1 mA	BLE event, timer, input	BLE connected but app is not interacting; input driver at low sample rate.
Advertising	~50 μ A	BLE scan-req, timer, input	Between advertising intervals. CPU mostly off.
Deep sleep	~3 μ A	RTC, GPIO interrupt (button / capacitive)	Most peripherals off. Woken by RTC for the next advertising slot.

9.3 Sleep coordinator

The `power_thread` is responsible for requesting deeper power states. Every other thread reports 'activity' through a shared counter. When the counter is zero, the system can enter idle-connected; when zero and BLE is not connected, we can advertise at low rate; when zero and no advertising is in flight, we can deep-sleep until the next RTC-scheduled advertising slot.

9.4 Battery reporting

- Read state-of-charge from the PMIC every 5 minutes over I2C.
- Update the Status characteristic if the reading changes by ≥ 1 percentage point.
- At 15 %, queue a low_battery glow event. Once per hour until charged.
- At 5 %, begin refusing non-essential operations (signature still works; glow is deferred).
- At 1 %, go to deep sleep with only a wake-on-charger GPIO interrupt.

10. Factory provisioning

At the factory, each device is attached to a test jig running a small provisioning tool. The tool writes the per-device values and issues the factory certificate.

10.1 What gets written

- OTP byte: tier (Premium 0x01 / Simple 0x02).
- OTP 16 bytes: device serial number (random, unique, logged by ops at issuance).
- Flash (one-time slot): 32-byte per-device PBKDF2 salt.
- PSA Crypto: generate the Ed25519 attestation keypair in CryptoCell. The public key is read back.
- Flash: the factory certificate signed by the factory CA, binding (serial, tier, pubkey, timestamp).
- MCUboot rollback counter: initialised to 1.

10.2 Factory firmware vs. production firmware

We ship two firmware images to the factory:

- `moatly-factory.hex` — runs the provisioning dance, then self-wipes and instructs the programmer to flash production firmware.
- `moatly-production.hex` — the real firmware, signed by the OTA signing key, flashed after provisioning.

The factory firmware is never signed by the OTA key. The bootloader accepts it only when the device has a special OTP bit set indicating 'pre-provisioning.' Once provisioning is complete the bit is cleared and only production firmware can boot.

10.3 Factory CA handling

- Factory CA private key lives on an air-gapped signing HSM at the factory, with two-of-two custody (Moatly's ops lead and the factory's responsible person).
- Each device cert is issued on that HSM, not on the test jig directly.
- Revocation list is maintained by Moatly's ops and pushed to the backend monthly (or immediately on known compromise).

10.4 Quality control

- Test jig runs a brief self-test after provisioning: BLE advertises, attestation characteristic returns the new pubkey + cert, a test challenge is signed and verified by the jig.
- Any failure flags the device for re-work, not shipment.
- 1% of devices (random) receive a more thorough soak test — 8 hours of BLE + glow + signature cycles — before shipment.

11. Cross-cutting concerns

11.1 Memory budget

Resource	Budget	Current estimate
Flash (code)	256 KB	~180 KB (leaves room for a few OTAs without reshuffling partitions).
Flash (settings)	64 KB	~4 KB used (combination HMAC, retry counter, glow mute flag, battery calibration).
Flash (factory slot)	8 KB	~1 KB used (factory cert, salt, tier OTP mirror).
SRAM (application)	128 KB	~64 KB typical (BLE stack + PSA + app).
Stack (per thread)	2 KB to 8 KB	Sized per thread; validated by Zephyr's stack-canary checking.

11.2 Timing requirements

- Combination entry to signature response: <2 s p95 (the 500 ms PBKDF2 is the dominant term; we budget 1500 ms for everything else).
- Glow event receipt to visible LED change: <50 ms.
- BLE connect establishment from app initiation: <3 s.
- DFU full update of a 256 KB image: <2 min over BLE 5 1 Mbps PHY.

11.3 Firmware security posture

- **No JTAG / SWD access in production builds.** The debug access port is disabled via APPROTECT at first boot of production firmware. This is irreversible — a production device cannot be debugged without invasive hardware work.
- **No UART in production.** The UART pins are repurposed as GPIOs.
- **No telemetry.** Production firmware contains no network stack beyond BLE and makes no outbound calls.
- **No string literals for sensitive operations.** Logging is compiled out, so 'unlock attempt failed' etc. are not present in the flash image.
- **Constant-time compare for security-critical equality.** PSA provides constant-time primitives; we use them.

11.4 Observability (within the constraints)

Production firmware does not phone home. It does, however, maintain a tiny in-flash log of 'last N events' (event code only, no content) that is readable via BLE as a diagnostic characteristic when a debug flag is set via a signed-admin write. This is for field-diagnosis purposes and off by default.

The diagnostic characteristic is only present in firmware builds compiled with `CONFIG_MOATLY_FIELD_DIAGNOSTIC=y`, which is off in the shipping builds and may be enabled in a targeted release if a field issue needs it.

12. Testing

12.1 Unit tests

- Zephyr's ztest framework. Unit tests run on a native Linux target (POSIX port), not on hardware.
- Coverage targets: every function in src/combo, src/attest, src/glow/patterns has explicit tests.
- The combination vocabulary (§5.4) has property tests: a gesture stream with timings jittered by $\pm \text{bucket}/2$ always canonicalises to the same integer sequence.
- The PBKDF2 derivation is tested against known vectors.

12.2 Hardware-in-the-loop

A CI bench hosts 4 dev boards (2 Premium prototypes, 2 Simple prototypes) connected to a controlling PC. Tests are written in Python using bleak for BLE interactions.

Test	What it proves
PAIR-FW-1	Fresh device advertises. Central can read attestation characteristic; cert + pubkey returned.
CHALLENGE-FW-1	Correct combination on physical ring / button produces a valid Ed25519 signature verifying under the returned pubkey.
CHALLENGE-FW-2	Wrong combination produces a 0x02 'failed' notify; no signature is produced.
LOCKOUT-FW-1	After 5 failed attempts, the device reports <code>lockout_remaining_seconds</code> in status.
LOCKOUT-FW-2	After 20 failed attempts, device zeros its key; re-entering the old correct combination fails.
GLOW-FW-1	Glow characteristic writes produce the expected LED patterns (verified via a photodiode on the bench).
DFU-FW-1	Signed OTA of a dummy image installs and boots; version counter advances.
DFU-FW-2	Signed OTA with a lower rollback counter is refused by the bootloader.
DFU-FW-3	Malformed signature refuses at bootloader, stays on old firmware.
RECOVERY-FW-1	Corrupt primary slot → recovery firmware boots and accepts OTA.
POWER-FW-1	30-day soak: idle + 5/day combination entries consumes <85 % of battery.
RSSI-FW-1	Device wakes at -70 dBm approach and idles below -85 dBm with 10s debounce.

12.3 Security tests

- Fuzz the GATT characteristics: write malformed data, oversize values, rapid sequential writes. Expect no crashes, no memory leaks, no state corruption.

- Verify the constant-time compare (§11.3) via a timing-side-channel measurement on the bench.
- Attempt to extract the signing key via PSA API misuse (calling export on a non-exportable key, attempting to read CryptoCell-backed storage). All attempts should fail with `PSA_ERROR_NOT_PERMITTED`.
- Inject invalid factory certs; expect the device to still boot but refuse to sign (because the cert load fails after provisioning).

12.4 External audit coverage

The firmware is subject to the Phase 4 external audit per brief §10.5. Scope:

- Code review of `src/attest`, `src/combo`, `src/dfu`.
- Review of the PSA Crypto key-usage policies.
- Review of the MCUboot configuration and rollback handling.
- Review of factory provisioning handling the CA key.
- Penetration test: attempt key extraction via JTAG (should fail — APPROTECT), via power-analysis side-channel (out of scope for launch; called out as future work).

13. Implementation plan

The firmware timeline runs parallel to but slightly ahead of the Moat app Phase 3 timeline. Firmware on dev boards must be ready by Week 2 of Phase 3 so that the app's BLE bring-up has a real target. Production-grade firmware is required by Week 10 of Phase 3 for the 100-person cohort hardware.

13.1 Weekly plan (12 weeks)

Week	Firmware engineer
1	Repo scaffold. west workspace. Zephyr 3.7 LTS. nRF Connect SDK integration. First build runs on a dev board; RTT logging visible.
2	Device tree for both tiers. Tier detection (§3.5). BLE stack up; basic advertising + GATT service with a placeholder characteristic. Delivered to app team for first BLE integration.
3	Attestation characteristic: factory cert structure, PSA key generation, public-key read. No signing yet — just expose the pubkey.
4	Challenge + signature path. PSA sign-hash end-to-end. Without combination verification yet (always unlocks).
5	Input drivers. capture_button.c for Simple. Zephyr CAPSENSE bring-up for Premium. Both produce gesture streams.
6	Combination vocabulary + derivation. PBKDF2 → unlock-HMAC → PSA unlock. Lockout state machine. Real verification path replaces the always-unlock stub.
7	Glow driver. driver_ring.c (WS2812 via SPI DMA) and driver_led.c (PWM). Five pattern step functions.
8	MCUboot configuration. First signed image builds. DFU flow on dev boards (SMP handlers active).
9	Power management. Sleep coordinator, advertising cadence adjustments, PMIC battery read. First battery-life measurement.
10	Factory provisioning tool. Tier OTP, salt generation, cert issuance flow (against a test factory CA).
11	HIL test suite. Hardware bench setup. All PAIR-FW / CHALLENGE-FW / LOCKOUT-FW / GLOW-FW / DFU-FW tests passing.
12	Production-candidate firmware. APPROTECT enabled. Final signing. Release-candidate handed to ops for the 100-person cohort batch.

13.2 Dependencies on other teams

- Hardware team: dev boards by start of Week 1; Premium prototype hardware by Week 4; Simple prototype hardware by Week 5; 100-unit production batch available for flashing at Week 12.
- Mobile team: Phase 3 §4 BLE pairing code ready against our firmware by Week 3.
- Backend team: device enrolment + firmware manifest endpoints by Week 7.
- Security engineer: factory CA setup + custody procedure by Week 9.

- Designer: glow pattern visual review by Week 7; timing & palette confirmed by Week 8.

14. Open questions

Question	Owner	Needed by	Default if no answer
Confirm nRF5340 over nRF52840 at final BOM.	Hardware + security	Week 1	nRF5340 per §2.2.
Capacitive electrode count — 8 or 12?	Designer + hardware	Week 2	8 electrodes (simpler firmware, enough vocabulary).
PBKDF2 iteration count final — 100k or 200k?	Security	Week 6	100k (balances user-felt latency with cost-to-brute-force).
Rollback counter storage — NVS settings or OTP?	Security	Week 8	NVS settings, protected area. Moving to OTP uses OTP budget.
Factory CA HSM vendor.	Ops + security	Week 10	YubiHSM 2 or equivalent FIPS 140-2 L3.
Battery-life target — still 30 days with real usage data?	Hardware + firmware	Week 9	30 days measured at target usage profile; tune if short.
Glow amber colour — exact RGB hex.	Designer	Week 7	#FFA500 as a starting point; designer tunes visually.
Diagnostic characteristic (§11.4) — ship on by default or off by default?	Security + lead	Week 10	Off by default; ship-on requires a signed admin write.

15. Post-launch lifecycle

15.1 What the ongoing shape looks like

- Firmware OTA releases quarterly unless a security issue accelerates them.
- Every OTA release is signed on the air-gapped host and goes through two-engineer approval before publishing.
- Every release's hash is logged to the portal's release-notes page; users can verify their device has the version they think.

15.2 Forensics and incident response

If a device is returned for analysis (user reports a crash, a suspected compromise, or an unexpected behaviour):

- Read diagnostic characteristic via a signed-admin BLE write (if enabled on that device). Contains the last N event codes.
- If the device is wedged, trigger recovery firmware, flash a diagnostic build, extract runtime state.
- No user content ever flows through any of this. The diagnostic data is event codes only. A forensic analyst cannot reconstruct what the user did in their Moat from firmware diagnostics.

15.3 Key rotation

- Factory CA: rotated annually or on any compromise. New CA's public key is shipped in a firmware + app release; devices signed by the old CA continue to verify until that CA is revoked.
- OTA signing key: rotated every 2 years or on compromise. New key embedded in bootloader via a signed firmware release.
- Device attestation key: never rotates during device lifetime. A replaced device gets a fresh key via re-pair + re-enrolment.

15.4 Revocation

If a device is reported stolen or suspected compromised:

- App triggers device revocation via the backend `/v1/devices/revoke` endpoint (Phase 3 §4.5 + backend §8.2).
- Backend adds device pubkey to the revocation list.
- On the device's next successful handshake, the backend returns a 'revoked' response; the app triggers the firmware to zero its key (via a dedicated admin BLE write signed by the revocation authority).
- If the device never connects again, the revocation still prevents it from authenticating — the firmware's key is irrelevant to the backend's decision to refuse.

15.5 Closing

The firmware is where Moatly's cryptographic promises meet physical reality. Every abstract principle in the Build Brief — 'the device is the key,' 'attestation prevents Sybil attacks,' 'the user's combination is a local secret' — lives as a few hundred lines of C in this codebase. Keeping that code small, auditable, and boringly correct is the firmware engineer's ongoing job.

Build it small. Keep it signed. Let the CryptoCell do the heavy lifting. Resist the urge to add clever features.

— *End of Firmware Technical Breakdown* —

Moatly · Firmware technical breakdown · April 2026 · v0.1